FIAP | Engenharia de Software — 3ESR

CP1 — Avaliação Prática

Cadastro de Produtos com Busca de CEP e Persistência JSON

Identificação do Aluno

Nome:	
RM:	
Turma:	
Data:	

Informações da Avaliação

Disciplina: C# e o Ecossistema .NET
Professor: Rafael
Curso: 3ESR — Engenharia de Software
Tipo: Avaliação prática individual — Visual Studio 2022
Entrega: Upload na plataforma FIAP em formato .docx com evidências
Valor: 10,0 pontos

Regras da Avaliação

- A avaliação é INDIVIDUAL. Qualquer cópia resultará em nota ZERO para ambos.
- Todo o desenvolvimento deve ser feito no Visual Studio ou Visual Code
- Este documento deve ser preenchido com screenshots e código como evidência.
- Cada exercício indica quais evidências são obrigatórias.
- Screenshots devem mostrar claramente o resultado (compile e execute antes de capturar).
- Código deve ser colado como TEXTO (não como imagem), para que possa ser verificado.
- Entregue este arquivo .docx preenchido na plataforma FIAP até a data limite.

> ATENÇÃO: Evidência ausente ou ilegível = pontuação ZERO no critério correspondente. Capriche nos screenshots!

Critérios de Avaliação (Rubrica)

Critério	O que será avaliado	Pontos
Estrutura do Projeto	Projeto Windows Forms criado corretamente, organização de arquivos, classes em arquivos separados	**1,0**
Classes Modelo	Produto.cs e Endereco.cs com propriedades corretas e JsonPropertyName	**1,5**
Design do Form	Controles nomeados corretamente, layout organizado, labels descritivos	**1,0**
Validações	Campos obrigatórios, formato de preço, CEP com 8 dígitos	**1,5**
Consumo da ViaCEP	HttpClient, async/await, deserialização do JSON, preenchimento automático	**2,0**
Persistência JSON	Salvar (Serialize) e Carregar (Deserialize) em arquivo .json	**1,5**
ListView	Exibição dos dados, carregamento ao abrir, exclusão funcional	**1,0**
Evidências	Screenshots claros, código legível, documento organizado	**0,5**
	TOTAL	**10,0**

Cenário da Avaliação

Você foi contratado como desenvolvedor júnior em uma loja de eletrônicos. Sua primeira tarefa é criar um sistema desktop em Windows Forms para cadastro de produtos. O sistema deve permitir cadastrar produtos com informações do fornecedor (buscando o endereço automaticamente pelo CEP usando a API ViaCEP), salvar os dados em arquivo JSON e listar todos os produtos cadastrados.

> O sistema NÃO precisa de tela de login. Foque apenas no formulário de cadastro de produtos com todas as funcionalidades pedidas abaixo.

Exercício 1 — Criar o Projeto (1,0 ponto)

Crie um novo projeto Windows Forms no Visual Studio 2022 com as seguintes configurações:

- **Nome do projeto:** CadastroProdutos
- **Framework:** .NET 8
- **Tipo:** Windows Forms App

Após criar, renomeie o Form1.cs para FormPrincipal.cs (botão direito > Rename no Solution Explorer). Lembre-se de atualizar o Program.cs se necessário.

Evidências obrigatórias:

📸 **Screenshot do Solution Explorer mostrando o projeto criado e o FormPrincipal.cs**

(Cole aqui o screenshot ou o código)

Exercício 2 — Classes Modelo (1,5 pontos)

Crie duas classes em arquivos separados:

Classe Produto.cs:

- Nome (string)
- Preco (decimal)
- Quantidade (int)
- CepFornecedor (string)
- Logradouro (string)
- Bairro (string)
- Cidade (string)
- Estado (string)

Classe Endereco.cs:

Classe para deserializar o retorno da API ViaCEP. Deve conter os atributos [JsonPropertyName] para mapear os campos do JSON da API (cep, logradouro, bairro, localidade, uf).

> Dica: lembre-se que o campo "cidade" na ViaCEP se chama "localidade" e o campo "estado" se chama "uf". Use [JsonPropertyName] para fazer o mapeamento correto.

Evidências obrigatórias:

📸 **Código completo da classe Produto.cs (cole como texto)**

(Cole aqui o screenshot ou o código)

📸 **Código completo da classe Endereco.cs (cole como texto)**

(Cole aqui o screenshot ou o código)

Exercício 3 — Design do FormPrincipal (1,0 ponto)

Monte o layout do FormPrincipal com os seguintes controles. Siga a convenção de nomenclatura:

Controle	(Name)	Observação
`TextBox`	**`txtNome`**	Nome do produto
`TextBox`	**`txtPreco`**	Preço (decimal)
`TextBox`	**`txtQuantidade`**	Quantidade (inteiro)
`TextBox`	**`txtCep`**	CEP do fornecedor — MaxLength = 9
`Button`	**`btnBuscarCep`**	Text = "Buscar CEP"
`TextBox`	**`txtLogradouro`**	ReadOnly = true
`TextBox`	**`txtBairro`**	ReadOnly = true
`TextBox`	**`txtCidade`**	ReadOnly = true
`TextBox`	**`txtEstado`**	ReadOnly = true
`Button`	**`btnCadastrar`**	Text = "Cadastrar Produto"
`Button`	**`btnExcluir`**	Text = "Excluir Selecionado"
`ListView`	**`listViewProdutos`**	View = Details, FullRowSelect = true, GridLines = true
`Label`	**`lblStatus`**	Text vazio — para mensagens de feedback

Colunas da ListView (propriedade Columns):

Text (Header)	Width
Nome	180
Preço	100
Qtd	60
CEP	90
Cidade	130
Estado	60

Evidências obrigatórias:

📸 **Screenshot do FormPrincipal no Designer (mostrando o layout com todos os controles)**

(Cole aqui o screenshot ou o código)

Exercício 4 — Validações (1,5 pontos)

No evento Click do botão btnCadastrar, implemente as seguintes validações ANTES de cadastrar o produto:

- **Nome:** não pode estar vazio.
- **Preço:** deve ser um número decimal válido e maior que zero. Use decimal.TryParse() para converter.
- **Quantidade:** deve ser um número inteiro válido e maior que zero. Use int.TryParse().
- **CEP:** deve ter sido buscado (verifique se txtCidade.Text não está vazio).

Se alguma validação falhar, exiba mensagem no lblStatus com cor vermelha (Color.Red) e interrompa o cadastro com return.

> Dica: para mostrar feedback colorido, use: lblStatus.Text = "mensagem"; lblStatus.ForeColor = Color.Red;

Evidências obrigatórias:

📸 Código completo do método btnCadastrar_Click (cole como texto)

(Cole aqui o screenshot ou o código)

📸 Screenshot do app rodando com uma mensagem de erro de validação aparecendo

(Cole aqui o screenshot ou o código)

Exercício 5 — Consumo da API ViaCEP (2,0 pontos)

Implemente a busca de CEP no evento Click do botão btnBuscarCep. O método deve:

- Ser async (assinatura: private async void btnBuscarCep_Click)
- Limpar o CEP removendo traços e espaços
- Validar se o CEP possui exatamente 8 dígitos numéricos
- Usar HttpClient para chamar: https://viacep.com.br/ws/{cep}/json/
- Usar await http.GetStringAsync(url) para obter o JSON
- Deserializar com JsonSerializer.Deserialize<Endereco>(json)
- Preencher os TextBoxes de endereço (txtLogradouro, txtBairro, txtCidade, txtEstado)
- Tratar erros com try/catch (HttpRequestException)
- Mostrar feedback no lblStatus (verde para sucesso, vermelho para erro)

> IMPORTANTE: Não esqueça dos usings necessários: System.Net.Http e System.Text.Json

Evidências obrigatórias:

📸 Código completo do método btnBuscarCep_Click (cole como texto)

(Cole aqui o screenshot ou o código)

📸 Screenshot do app rodando após buscar o CEP 01310-100 (Av. Paulista) com campos preenchidos

(Cole aqui o screenshot ou o código)

📸 Screenshot do app rodando com CEP inválido mostrando mensagem de erro

(Cole aqui o screenshot ou o código)

Exercício 6 — Persistência em JSON (1,5 pontos)

Implemente a persistência dos produtos em arquivo JSON. Você deve criar:

1. Variáveis de instância no FormPrincipal:

- private List<Produto> produtos = new List<Produto>();
- private readonly string caminhoArquivo = "produtos.json";

2. Método SalvarDados():

- Serializa a lista produtos para JSON usando JsonSerializer.Serialize()
- Grava no arquivo usando File.WriteAllText()

3. Método CarregarDados():

- Verifica se o arquivo existe com File.Exists()
- Lê o conteúdo com File.ReadAllText()
- Deserializa com JsonSerializer.Deserialize<List<Produto>>()
- Popula a ListView com os dados carregados

4. Chamadas:

- CarregarDados() deve ser chamado no construtor do Form (após InitializeComponent)
- SalvarDados() deve ser chamado ao cadastrar e ao excluir

Evidências obrigatórias:

📸 **Código dos métodos SalvarDados() e CarregarDados() (cole como texto)**

(Cole aqui o screenshot ou o código)

📸 **Screenshot: cadastre 2 produtos, FECHE o app, REABRA e mostre que os dados persistiram na ListView**

(Cole aqui o screenshot ou o código)

Exercício 7 — ListView e Exclusão (1,0 ponto)

Implemente:

1. Método AdicionarNaListView(Produto p):

- Cria um ListViewItem com o Nome do produto
- Adiciona SubItems: Preço (formatado como R$ 0,00), Quantidade, CEP, Cidade, Estado
- Adiciona o item na listViewProdutos

2. Botão Excluir (btnExcluir_Click):

- Verifica se há item selecionado na ListView
- Exibe MessageBox.Show de confirmação (Yes/No)
- Remove o item da lista produtos por índice (SelectedIndices[0])
- Remove o item da ListView
- Chama SalvarDados() para atualizar o arquivo JSON

Evidências obrigatórias:

📸 Código do método AdicionarNaListView (cole como texto)

(Cole aqui o screenshot ou o código)

📸 Código do método btnExcluir_Click (cole como texto)

(Cole aqui o screenshot ou o código)

📸 Screenshot mostrando o MessageBox de confirmação de exclusão

(Cole aqui o screenshot ou o código)

Checklist Final

Antes de entregar, verifique se você completou todas as etapas:

- ☐ Projeto CadastroProdutos criado em .NET 8 Windows Forms
- ☐ Produto.cs criado com todas as 8 propriedades
- ☐ Endereco.cs criado com [JsonPropertyName] corretos
- ☐ FormPrincipal com todos os controles nomeados corretamente
- ☐ Validação de nome, preço (decimal), quantidade (int) e CEP buscado
- ☐ Busca de CEP funcional com HttpClient + async/await
- ☐ SalvarDados() com JsonSerializer.Serialize + File.WriteAllText
- ☐ CarregarDados() com File.ReadAllText + Deserialize<List<Produto>>
- ☐ CarregarDados() chamado no construtor do Form
- ☐ ListView mostrando dados com colunas corretas
- ☐ Exclusão com confirmação + SalvarDados()
- ☐ Dados persistem após fechar e reabrir o app
- ☐ Todos os screenshots claros e legíveis
- ☐ Código colado como TEXTO (não imagem)

> Entregue este arquivo .docx preenchido na plataforma FIAP. Boa prova!

"Código bom é código que funciona, é testado e outro dev entende."
— Prof. Rafael